

Compute-aware Routing

Abstract—This document is a model and instructions for L^AT_EX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert

In this paper, we investigate a failure-aware compute routing problem in geo-distributed cloud networks. Different from conventional traffic engineering, which optimizes traffic allocation for fixed source–destination pairs, compute routing inherently couples task placement with network routing. Given a set of tasks with both computing and communication requirements, the system must jointly determine the execution location of each task and the routing strategy for its input and output data over the wide-area network. Accordingly, we formulate a joint task placement and multipath traffic allocation model, and incorporate Conditional Value-at-Risk (CVaR) as a tail-risk metric to enhance service reliability under failure scenarios.

I. PROBLEM FORMULATION

This section first formulates the failure-aware compute routing problem, which jointly optimizes availability and resource balancing, and establishes a single-level joint optimization model as a complete benchmark. Considering that task placement and traffic allocation typically operate at different timescales in practical deployments, we further develop a two-level task-placement and traffic-allocation approximation model to approximate the performance of the single-level benchmark while reducing the solving complexity and online re-optimization overhead. The main notations used in this section are summarized in Table I.

A. System Model

In the failure-aware compute routing framework, the topology information, task requirements, candidate path sets, and failure probability information jointly constitute the model inputs. Based on these inputs, the system needs to jointly determine task execution-node placement and multipath traffic allocation.

Topology information. The geo-distributed cloud network is modeled as a directed graph $G = (V, E)$, where V denotes the set of nodes and E denotes the set of directed links. Let $\mathcal{M} \subseteq V$ denote the set of compute-capable nodes, and let $\mathcal{R} = V \setminus \mathcal{M}$ denote the set of forwarding-only nodes. For each compute node $m \in \mathcal{M}$, its computing resource capacity is represented by a d -dimensional vector. The capacity of node m on the k -th resource dimension is denoted by $C_m^{(k)}$, where $k = 1, \dots, d$. For each directed link $e \in E$, its bandwidth capacity is denoted by B_e .

TABLE I
SUMMARY OF NOTATIONS

Topology-related Notation	
$G = (V, E)$	Directed network topology with node set V and directed link set E .
$\mathcal{M} \subseteq V$	Set of compute-capable nodes.
$C_m \in \mathbb{R}_+^d$	d -dimensional resource capacity vector of compute node m .
B_e	Bandwidth capacity of link e .
$\delta_{e,p} \in \{0, 1\}$	Path–link incidence indicator; $\delta_{e,p} = 1$ if link e belongs to path p .
Task-related Notation	
$\mathcal{J}$	Set of tasks.
s_i, d_i	Source node and destination node of task i .
$w_i \in \mathbb{R}_+^d$	d -dimensional compute-resource demand of task i .
$\mathcal{M}_i \subseteq \mathcal{M}$	Set of candidate execution nodes for task i .
$b_i^{\text{in}}, b_i^{\text{out}}$	Input data size and output data size of task i .
$\mathcal{P}_{i,m}^{\text{in}}$	Candidate input paths from source node s_i to execution node m .
$\mathcal{P}_{i,m}^{\text{out}}$	Candidate output paths from execution node m to destination node d_i .
Failure-scenario Notation	
S	Set of failure scenarios.
$p_s \in [0, 1]$	Occurrence probability of failure scenario s .
$\eta_m(s) \in \{0, 1\}$	Compute-node availability indicator; $\eta_m(s) = 1$ if node m is available in scenario s .
$\chi_p(s) \in \{0, 1\}$	Path availability indicator; $\chi_p(s) = 1$ if path p is available in scenario s .
$D_{i,s}^{\text{in}}, D_{i,s}^{\text{out}} \geq 0$	Delivered input and output traffic of task i in failure scenario s .
$l_{i,s} \in [0, 1]$	Normalized loss of task i in failure scenario s .
$\ell_s \in [0, 1]$	System-level loss in failure scenario s .
$\alpha \in [0, 1], u_s \geq 0$	Auxiliary variables for CVaR linearization.
$\beta \in (0, 1)$	Confidence level of the CVaR risk metric.
Decision Variables	
$y_{i,m} \in \{0, 1\}$	Task placement variable; $y_{i,m} = 1$ if task i is assigned to compute node m .
$x_{i,m,p}^{\text{in}} \geq 0$	Input traffic allocated on path $p \in \mathcal{P}_{i,m}^{\text{in}}$ when task i is assigned to node m .
$x_{i,m,q}^{\text{out}} \geq 0$	Output traffic allocated on path $q \in \mathcal{P}_{i,m}^{\text{out}}$ when task i is assigned to node m .
Objective-related Variables	
CVaR_β	CVaR-based tail-risk metric under confidence level β .
$U_{\text{node}}^{\text{max}} \in [0, 1]$	Maximum compute-node utilization.
$U_{\text{link}}^{\text{max}} \in [0, 1]$	Maximum pre-allocated link utilization in the normal state.
$\rho, \lambda \in [0, 1]$	Trade-off coefficients in the objective function.

Task information. Let $\mathcal{J}$ denote the set of tasks. For each task $i \in \mathcal{J}$, $s_i \in V$ and $d_i \in V$ denote its source node and destination node, respectively, and $w_i^{(k)}$ denotes its demand on the k -th compute-resource dimension. The set of candidate execution nodes for task i is denoted by $\mathcal{M}_i \subseteq \mathcal{M}$. If task i is assigned to a candidate node $m \in \mathcal{M}_i$, its communication

process consists of two stages: the input data are transferred from the source node s_i to the execution node m , and the output data are transferred from the execution node m to the destination node d_i . The corresponding input and output data sizes are denoted by b_i^{in} and b_i^{out} , respectively.

Path information. For each task–candidate-node pair (i, m) , let $\mathcal{P}_{i,m}^{\text{in}}$ denote the set of candidate input paths, and let $\mathcal{P}_{i,m}^{\text{out}}$ denote the set of candidate output paths. In this model, each task is indivisible, i.e., each task can be assigned to only one execution node. In contrast, traffic is splittable, meaning that both the input traffic and the output traffic can be distributed over multiple paths in their respective candidate path sets.

For any link e and input path $p \in \mathcal{P}_{i,m}^{\text{in}}$, we define the path–link incidence indicator $\delta_{e,p} \in \{0, 1\}$. Specifically, $\delta_{e,p} = 1$ if link e belongs to path p , and $\delta_{e,p} = 0$ otherwise. Similarly, $\delta_{e,q} \in \{0, 1\}$ is defined for each output path $q \in \mathcal{P}_{i,m}^{\text{out}}$.

Failure probability information. The failure model consists of a set of basic failure events Z , where each event $z \in Z$ can represent the failure of a link, a node, or a shared risk group. The occurrence probability of event z is denoted by p_z , which can be estimated from historical failure data. Based on the occurrence states of all failure events, we define a network-state scenario as $s = (s_z)_{z \in Z}$, where $s_z \in \{0, 1\}$ indicates whether failure event z occurs. The set of all possible network-state scenarios is denoted by S . Under the assumption that basic failure events are independent, the occurrence probability of scenario $s \in S$ is given by

$$p_s = \prod_{z \in Z} (s_z p_z + (1 - s_z)(1 - p_z)). \quad (1)$$

These scenario probabilities satisfy

$$\sum_{s \in S} p_s = 1, \quad p_s \geq 0. \quad (2)$$

For any input path $p \in \mathcal{P}_{i,m}^{\text{in}}$, we define the path availability indicator $\chi_p(s) \in \{0, 1\}$. If all links on path p are available under scenario s , then $\chi_p(s) = 1$; otherwise, $\chi_p(s) = 0$. Similarly, $\chi_q(s) \in \{0, 1\}$ is defined for each output path $q \in \mathcal{P}_{i,m}^{\text{out}}$. We also define the compute-node availability indicator $\eta_m(s) \in \{0, 1\}$, which indicates whether execution node m is available under scenario s .

Variables. Failure-aware compute routing requires jointly determining the task execution locations and the two-stage multipath traffic allocation. Let $y_{i,m} \in \{0, 1\}$ denote the task placement variable. If task i is assigned to compute node m , then $y_{i,m} = 1$; otherwise, $y_{i,m} = 0$. Let $x_{i,m,p}^{\text{in}} \geq 0$ denote the input traffic pre-allocated to input path $p \in \mathcal{P}_{i,m}^{\text{in}}$ in the normal state when task i is assigned to node m . Similarly, let $x_{i,m,q}^{\text{out}} \geq 0$ denote the output traffic pre-allocated to output path $q \in \mathcal{P}_{i,m}^{\text{out}}$ in the normal state. In this paper, each task can select only one execution node, while its input and output traffic can be split over their respective candidate path sets.

B. Optimization Metrics

We introduce three optimization metrics: (i) the maximum node utilization, (ii) the maximum link utilization, and (iii) the

CVaR-based tail failure risk. The first two metrics characterize the load balancing of computing and network resources, while the third metric captures the service availability risk under failure scenarios.

Maximum node utilization. Let $U_{\text{node}}^{\text{max}} \in [0, 1]$ denote the maximum node utilization. Each task can be assigned to exactly one candidate execution node, i.e.,

$$\sum_{m \in \mathcal{M}_i} y_{i,m} = 1, \quad \forall i \in \mathcal{I}. \quad (3)$$

The tasks placed on compute nodes consume multi-dimensional computing resources. It follows:

$$\sum_{i \in \mathcal{I}: m \in \mathcal{M}_i} y_{i,m} w_i^{(k)} \leq C_m^{(k)} U_{\text{node}}^{\text{max}}, \quad \forall m \in \mathcal{M}, \quad k = 1, \dots, d. \quad (4)$$

Maximum link utilization. Let $U_{\text{link}}^{\text{max}} \in [0, 1]$ denote the maximum pre-allocated link utilization in the normal state. To improve robustness under failure scenarios, we allow each task to be pre-allocated with bandwidth beyond its basic demand, while traffic can only be allocated over the candidate paths associated with the selected execution node. Thus, the input and output traffic allocations should satisfy

$$\sum_{p \in \mathcal{P}_{i,m}^{\text{in}}} x_{i,m,p}^{\text{in}} \geq b_i^{\text{in}} y_{i,m}, \quad \forall i \in \mathcal{I}, \quad m \in \mathcal{M}_i, \quad (5)$$

$$\sum_{q \in \mathcal{P}_{i,m}^{\text{out}}} x_{i,m,q}^{\text{out}} \geq b_i^{\text{out}} y_{i,m}, \quad \forall i \in \mathcal{I}, \quad m \in \mathcal{M}_i. \quad (6)$$

Let $\bar{M}_{i,m,p}^{\text{in}}$ and $\bar{M}_{i,m,q}^{\text{out}}$ denote the upper bounds on the traffic allocated to input path p and output path q , respectively. The traffic allocation variables and the task placement variable are coupled by

$$x_{i,m,p}^{\text{in}} \leq \bar{M}_{i,m,p}^{\text{in}} y_{i,m}, \quad \forall i \in \mathcal{I}, \quad m \in \mathcal{M}_i, \quad p \in \mathcal{P}_{i,m}^{\text{in}}, \quad (7)$$

$$x_{i,m,q}^{\text{out}} \leq \bar{M}_{i,m,q}^{\text{out}} y_{i,m}, \quad \forall i \in \mathcal{I}, \quad m \in \mathcal{M}_i, \quad q \in \mathcal{P}_{i,m}^{\text{out}}. \quad (8)$$

For each link $e \in E$, the pre-allocated link load in the normal state is defined as

$$L_e = \sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}_i} \left(\sum_{p \in \mathcal{P}_{i,m}^{\text{in}}} \delta_{e,p} x_{i,m,p}^{\text{in}} + \sum_{q \in \mathcal{P}_{i,m}^{\text{out}}} \delta_{e,q} x_{i,m,q}^{\text{out}} \right). \quad (9)$$

Accordingly, the maximum link utilization should satisfy

$$L_e \leq B_e U_{\text{link}}^{\text{max}}, \quad \forall e \in E. \quad (10)$$

CVaR-based failure-risk metric. Inspired by TeaVaR, we employ CVaR to characterize the tail service loss under probabilistic failure scenarios. Given a task placement and a traffic allocation, the delivered input and output traffic of task i in scenario s are defined as

$$D_{i,s}^{\text{in}} = \sum_{m \in \mathcal{M}_i} \sum_{p \in \mathcal{P}_{i,m}^{\text{in}}} \eta_m(s) \chi_p(s) x_{i,m,p}^{\text{in}}, \quad (11)$$

$$D_{i,s}^{\text{out}} = \sum_{m \in \mathcal{M}_i} \sum_{q \in \mathcal{P}_{i,m}^{\text{out}}} \eta_m(s) \chi_q(s) x_{i,m,q}^{\text{out}}. \quad (12)$$

Since task service depends on computation at the execution node as well as the two-stage input/output transmission, the unavailability of any stage may lead to service degradation. We define the normalized task-level loss $l_{i,s}$ as

$$l_{i,s} = \max \left\{ 1 - \frac{D_{i,s}^{\text{in}}}{b_i^{\text{in}}}, 1 - \frac{D_{i,s}^{\text{out}}}{b_i^{\text{out}}}, 0 \right\}, \quad \forall i \in \mathcal{J}, s \in S. \quad (13)$$

The system-level loss in each scenario is defined as the maximum loss among all tasks, which prevents the model from reducing only the average loss while sacrificing the availability of individual tasks:

$$\ell_s \geq l_{i,s}, \quad \forall i \in \mathcal{J}, s \in S. \quad (14)$$

Given a risk confidence level $\beta \in (0, 1)$, we introduce the VaR auxiliary variable α and the CVaR linearization variables u_s to obtain a linear representation of CVaR:

$$u_s \geq \ell_s - \alpha, \quad u_s \geq 0, \quad \forall s \in S. \quad (15)$$

Therefore, the CVaR-based tail-risk metric is written as

$$\text{CVaR}_\beta = \alpha + \frac{1}{1-\beta} \sum_{s \in S} p_s u_s. \quad (16)$$

Here, α represents the linearized VaR threshold, and CVaR_β represents the expected tail failure loss beyond this threshold.

C. Single-Level Joint Optimization Model

Based on the system model and the optimization metrics, we first formulate a single-level joint optimization model as a complete performance benchmark. This model jointly determines the task execution-node placement and the input/output multipath traffic allocation within a unified optimization problem.

The single-level joint optimization model is formulated as

follows:

$$\begin{aligned} \mathbf{P}_0 : \quad & \min \rho \text{CVaR}_\beta + (1-\rho) [\lambda U_{\text{link}}^{\text{max}} + (1-\lambda) U_{\text{node}}^{\text{max}}] \\ & \begin{cases} \sum_{m \in \mathcal{M}_i} y_{i,m} = 1, & \forall i \in \mathcal{J}, \\ \sum_{i \in \mathcal{J}: m \in \mathcal{M}_i} y_{i,m} w_i^{(k)} \leq C_m^{(k)} U_{\text{node}}^{\text{max}}, & \forall m \in \mathcal{M}, k = 1, \dots, d, \\ \sum_{p \in \mathcal{P}_{i,m}^{\text{in}}} x_{i,m,p}^{\text{in}} \geq b_i^{\text{in}} y_{i,m}, & \forall i \in \mathcal{J}, m \in \mathcal{M}_i, \\ \sum_{q \in \mathcal{P}_{i,m}^{\text{out}}} x_{i,m,q}^{\text{out}} \geq b_i^{\text{out}} y_{i,m}, & \forall i \in \mathcal{J}, m \in \mathcal{M}_i, \\ x_{i,m,p}^{\text{in}} \leq \bar{M}_{i,m,p}^{\text{in}} y_{i,m}, & \forall i, m, p \in \mathcal{P}_{i,m}^{\text{in}}, \\ x_{i,m,q}^{\text{out}} \leq \bar{M}_{i,m,q}^{\text{out}} y_{i,m}, & \forall i, m, q \in \mathcal{P}_{i,m}^{\text{out}}, \\ L_e \leq B_e U_{\text{link}}^{\text{max}}, & \forall e \in E, \\ D_{i,s}^{\text{in}} = \sum_{m \in \mathcal{M}_i} \sum_{p \in \mathcal{P}_{i,m}^{\text{in}}} \eta_m(s) \chi_p(s) x_{i,m,p}^{\text{in}}, & \forall i \in \mathcal{J}, s \in S, \\ D_{i,s}^{\text{out}} = \sum_{m \in \mathcal{M}_i} \sum_{q \in \mathcal{P}_{i,m}^{\text{out}}} \eta_m(s) \chi_q(s) x_{i,m,q}^{\text{out}}, & \forall i \in \mathcal{J}, s \in S, \\ l_{i,s} \geq 1 - \frac{D_{i,s}^{\text{in}}}{b_i^{\text{in}}}, \quad l_{i,s} \geq 1 - \frac{D_{i,s}^{\text{out}}}{b_i^{\text{out}}}, & \forall i \in \mathcal{J}, s \in S, \\ \ell_s \geq l_{i,s}, & \forall i \in \mathcal{J}, s \in S, \\ u_s \geq \ell_s - \alpha, \quad u_s \geq 0, & \forall s \in S, \\ \text{CVaR}_\beta = \alpha + \frac{1}{1-\beta} \sum_{s \in S} p_s u_s, \\ y_{i,m} \in \{0, 1\}, & \forall i \in \mathcal{J}, m \in \mathcal{M}_i, \\ x_{i,m,p}^{\text{in}} \geq 0, \quad x_{i,m,q}^{\text{out}} \geq 0, & \forall i, m, p, q, \\ 0 \leq l_{i,s} \leq 1, \quad 0 \leq \ell_s \leq 1, & \forall i \in \mathcal{J}, s \in S, \\ 0 \leq \alpha \leq 1, \quad 0 \leq U_{\text{node}}^{\text{max}} \leq 1, \quad 0 \leq U_{\text{link}}^{\text{max}} \leq 1. \end{cases} \end{aligned} \quad (17)$$

Here, $\rho \in [0, 1]$ balances the tail failure-risk objective and the resource-balancing objective, while $\lambda \in [0, 1]$ further balances the link utilization and the compute-node utilization. A larger ρ makes the model place more emphasis on service availability under failure scenarios, whereas a smaller ρ makes it focus more on resource load balancing in the normal state.

Model $\mathbf{P}_0$ captures the key coupling relationships in compute routing. However, it contains binary task placement variables, continuous traffic allocation variables, and scenario-dependent risk constraints. Therefore, its size grows rapidly with the number of tasks, candidate execution nodes, candidate paths, and failure scenarios. More importantly, in practical geo-distributed cloud systems, task placement and traffic allocation typically operate at different timescales. Task placement involves service migration, state initialization, and scheduling stability, and is thus usually updated at a relatively slow timescale. In contrast, traffic allocation mainly corresponds to bandwidth allocation and path-flow adjustment, which can be re-optimized at a minute-level or even shorter timescale according to network states and failure risks. Therefore, we further develop a two-level task-placement and traffic-allocation approximation model to approximate the performance of the single-level benchmark while reducing the solving complexity and online re-optimization overhead.

D. Two-Level Task-Placement and Traffic-Allocation Approximation

The core idea of the two-level approximation is to decouple task placement from traffic allocation. The task-placement

layer targets slow-timescale placement decisions and only optimizes the execution-node selection for tasks. The traffic-allocation layer targets fast-timescale traffic routing and further optimizes the input/output multipath traffic allocation given a task placement.

1) *Task-Placement Layer Model:*

a) *Base Placement Model:* The task-placement layer considers two basic optimization objectives: the maximum node utilization and the node-failure-risk CVaR, where the former is defined consistently with the single-level joint optimization model. Let S^N denote the set of node-failure scenarios, p_r^N denote the occurrence probability of scenario $r \in S^N$, and $\eta_m(r)$ denote the availability of node m in scenario r .

The task-level loss in the task-placement layer depends on whether the selected execution node is available:

$$l_{i,r}^N \geq 1 - \sum_{m \in \mathcal{M}_i} \eta_m(r) y_{i,m}, \quad l_{i,r}^N \geq 0. \quad (18)$$

The system-level node loss is defined as the maximum loss among all tasks:

$$\ell_r^N \geq l_{i,r}^N, \quad \forall i \in \mathcal{J}, r \in S^N. \quad (19)$$

Given the node-layer risk confidence level $\beta_N \in (0, 1)$, the node-layer CVaR risk is defined as

$$\text{CVaR}_{\beta_N}^N = \alpha^N + \frac{1}{1 - \beta_N} \sum_{r \in S^N} p_r^N u_r^N. \quad (20)$$

Given the node-risk weight $\rho_N \in [0, 1]$, the base placement model is formulated as follows:

$$\begin{aligned} \mathbf{P}_N : \quad & \min \rho_N \text{CVaR}_{\beta_N}^N + (1 - \rho_N) U_{\text{node}}^{\max} \\ \text{s.t.} \quad & \begin{cases} \sum_{m \in \mathcal{M}_i} y_{i,m} = 1, & \forall i \in \mathcal{J}, \\ \sum_{i \in \mathcal{J} : m \in \mathcal{M}_i} y_{i,m} w_i^{(k)} \leq C_m^{(k)} U_{\text{node}}^{\max}, & \forall m \in \mathcal{M}, k = 1, \dots, d, \\ l_{i,r}^N \geq 1 - \sum_{m \in \mathcal{M}_i} \eta_m(r) y_{i,m}, & \forall i \in \mathcal{J}, r \in S^N, \\ l_{i,r}^N \geq 0, & \forall i \in \mathcal{J}, r \in S^N, \\ \ell_r^N \geq l_{i,r}^N, & \forall i \in \mathcal{J}, r \in S^N, \\ u_r^N \geq \ell_r^N - \alpha^N, & \forall r \in S^N, \\ u_r^N \geq 0, & \forall r \in S^N, \\ y_{i,m} \in \{0, 1\}, & \forall i \in \mathcal{J}, m \in \mathcal{M}_i, \\ \alpha^N \geq 0, \\ 0 \leq U_{\text{node}}^{\max} \leq 1. \end{cases} \end{aligned} \quad (21)$$

Model $\mathbf{P}_N$ captures the basic quality requirements of the task-placement layer. It aims to balance node load while ensuring that the placement incurs a low tail loss under node-failure scenarios. Different from the single-level joint optimization model, $\mathbf{P}_N$ does not include actual traffic allocation variables, and thus its integer-programming scale is much smaller than that of $\mathbf{P}_0$.

b) *Network-Affinity Metric:* The base placement model only captures computing resources and node-failure risks, but lacks network awareness. As a result, it may generate placements that are unfavorable from the network perspective. For example, some placements may have low node utilization and low node-failure risk, but their input/output candidate paths may be few, highly overlapped, or long, making them more likely to create link bottlenecks in the traffic-allocation layer. Therefore, without introducing the actual traffic allocation variables x , we pre-compute a static network-affinity cost $\phi_{i,m}$ for each task-candidate-node pair (i, m) to guide the task-placement layer toward placements with better network structures.

For a path set $\mathcal{P}$, we define the path-count insufficiency cost, the path-overlap cost, and the path-length cost as follows:

$$c_{\text{cnt}}(\mathcal{P}) = \frac{1}{|\mathcal{P}|}, \quad (22)$$

$$c_{\text{ov}}(\mathcal{P}) = \max_{e \in E} \frac{|\{p \in \mathcal{P} : e \in p\}|}{|\mathcal{P}|}, \quad (23)$$

$$c_{\text{len}}(\mathcal{P}) = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \frac{|p|}{L_{\max}}. \quad (24)$$

Here, $L_{\max}$ denotes the maximum path length among all candidate paths. The composite network-affinity cost of path set $\mathcal{P}$ is defined as

$$c(\mathcal{P}) = \theta_1 c_{\text{cnt}}(\mathcal{P}) + \theta_2 c_{\text{ov}}(\mathcal{P}) + \theta_3 c_{\text{len}}(\mathcal{P}), \quad \theta_1 + \theta_2 + \theta_3 = 1. \quad (25)$$

The network-affinity cost of assigning task i to candidate node m is defined as

$$\phi_{i,m} = \frac{b_i^{\text{in}} c(\mathcal{P}_{i,m}^{\text{in}}) + b_i^{\text{out}} c(\mathcal{P}_{i,m}^{\text{out}})}{b_i^{\text{in}} + b_i^{\text{out}}}. \quad (26)$$

For a given placement y , the average network-affinity cost is defined as

$$A(y) = \frac{1}{|\mathcal{J}|} \sum_{i \in \mathcal{J}} \sum_{m \in \mathcal{M}_i} \phi_{i,m} y_{i,m}. \quad (27)$$

A smaller $A(y)$ indicates that the corresponding input/output candidate paths are richer, less overlapped, and shorter on average. We then use this metric to guide candidate placement generation in the task-placement layer.

c) *Candidate Placement Generation:* Based on the network-affinity metric, we further generate a set of candidate task placements instead of using only the single optimal solution obtained from the base placement model. Specifically, we first solve the base placement model $\mathbf{P}_N$ and use its maximum node utilization and node-failure risk as node-layer quality references. Then, under the condition that the node utilization and node risk are allowed to degrade only within prescribed tolerances, we minimize the average network-affinity cost $A(y)$ to guide the task-placement layer toward placements with richer candidate paths, lower path overlap, and shorter path lengths. To avoid generating duplicate placements, after each candidate is obtained, a no-good cut is added to exclude

Algorithm 1 Network-Affinity-Aware Candidate Placement Generation

- 1: **Input:** Base placement model $\mathbf{P}_N$, affinity cost $A(y)$, tolerances ϵ_U, ϵ_R , candidate budget R
 - 2: **Output:** Candidate placement set $\mathcal{Y}$
 - 3: **Step 1: Obtain the reference placement.**
 - 4: Solve $\mathbf{P}_N$ and obtain y^{ref} .
 - 5: Set $U_N^{\text{ref}} = U_{\text{node}}^{\text{max}}(y^{\text{ref}})$ and $R_N^{\text{ref}} = \text{CVaR}_{\beta_N}^N(y^{\text{ref}})$.
 - 6: **Step 2: Build quality constraints.**
 - 7: Define $\mathcal{Q}_N = \{U_{\text{node}}^{\text{max}}(y) \leq U_N^{\text{ref}} + \epsilon_U, \text{CVaR}_{\beta_N}^N(y) \leq R_N^{\text{ref}} + \epsilon_R\}$.
 - 8: Initialize $\mathcal{Y} \leftarrow \emptyset$ and $\mathcal{G} \leftarrow \emptyset$.
 - 9: **Step 3: Generate candidate placements.**
 - 10: **for** $r = 1$ to R **do**
 - 11: Solve $y^{(r)} \leftarrow \arg \min_y A(y)$ subject to $\mathcal{C}_N, \mathcal{Q}_N$, and all cuts in $\mathcal{G}$.
 - 12: **if** the problem is infeasible **then** s.t.
 - 13: **break**
 - 14: **end if**
 - 15: $\mathcal{Y} \leftarrow \mathcal{Y} \cup \{y^{(r)}\}$.
 - 16: Add the no-good cut $\sum_{i \in \mathcal{J}} y_{i,m_i^{(r)}} \leq |\mathcal{J}| - 1$ to $\mathcal{G}$.
 - 17: **end for**
 - 18: **Return** $\mathcal{Y}$.
-

the current placement from the feasible region in subsequent iterations. The candidate placement generation procedure is summarized in Algorithm 1, where R denotes the candidate budget. This procedure provides the traffic-allocation layer with a small number of candidate placements that have both acceptable task-placement-layer quality and favorable network structures.

2) *Traffic-Allocation Layer Model:* Given a candidate placement $\bar{y} \in \mathcal{Y}$ generated by the task-placement layer, the traffic-allocation layer fixes the placement variables and optimizes only the input/output multipath bandwidth pre-allocation variables x^{in} and x^{out} . This layer follows the link-utilization and CVaR-risk definitions in the single-level joint optimization model, and evaluates the performance of each candidate placement under the complete joint failure scenario set $\mathcal{S}$.

For a given $\bar{y}$, the traffic-allocation layer is formulated as

follows:

$$\begin{aligned}
 \mathbf{P}_L(\bar{y}) : \quad & F_L^*(\bar{y}) = \min \rho_L \text{CVaR}_{\beta_L}^L(\bar{y}, x) + (1 - \rho_L)U_{\text{link}}^{\text{max}} \\
 & \left\{ \begin{array}{ll} \sum_{p \in \mathcal{P}_{i,m}^{\text{in}}} x_{i,m,p}^{\text{in}} \geq b_i^{\text{in}} \bar{y}_{i,m}, & \forall i \in \mathcal{J}, m \in \mathcal{M}_i, \\ \sum_{q \in \mathcal{P}_{i,m}^{\text{out}}} x_{i,m,q}^{\text{out}} \geq b_i^{\text{out}} \bar{y}_{i,m}, & \forall i \in \mathcal{J}, m \in \mathcal{M}_i, \\ x_{i,m,p}^{\text{in}} \leq \bar{M}_{i,m,p}^{\text{in}} \bar{y}_{i,m}, & \forall i, m, p \in \mathcal{P}_{i,m}^{\text{in}}, \\ x_{i,m,q}^{\text{out}} \leq \bar{M}_{i,m,q}^{\text{out}} \bar{y}_{i,m}, & \forall i, m, q \in \mathcal{P}_{i,m}^{\text{out}}, \\ L_e(x) \leq B_e U_{\text{link}}^{\text{max}}, & \forall e \in E, \\ D_{i,s}^{\text{in}} = \sum_{m \in \mathcal{M}_i} \sum_{p \in \mathcal{P}_{i,m}^{\text{in}}} \eta_m(s) \chi_p(s) x_{i,m,p}^{\text{in}}, & \forall i \in \mathcal{J}, s \in S, \\ D_{i,s}^{\text{out}} = \sum_{m \in \mathcal{M}_i} \sum_{q \in \mathcal{P}_{i,m}^{\text{out}}} \eta_m(s) \chi_q(s) x_{i,m,q}^{\text{out}}, & \forall i \in \mathcal{J}, s \in S, \\ l_{i,s}^L \geq 1 - \frac{D_{i,s}^{\text{in}}}{b_i^{\text{in}}}, & \forall i \in \mathcal{J}, s \in S, \\ l_{i,s}^L \geq 1 - \frac{D_{i,s}^{\text{out}}}{b_i^{\text{out}}}, & \forall i \in \mathcal{J}, s \in S, \\ l_{i,s}^L \geq 0, & \forall i \in \mathcal{J}, s \in S, \\ \ell_s^L \geq l_{i,s}^L, & \forall i \in \mathcal{J}, s \in S, \\ u_s^L \geq \ell_s^L - \alpha^L, & \forall s \in S, \\ u_s^L \geq 0, & \forall s \in S, \\ \text{CVaR}_{\beta_L}^L(\bar{y}, x) = \alpha^L + \frac{1}{1 - \beta_L} \sum_{s \in S} p_s u_s^L, & \\ x_{i,m,p}^{\text{in}} \geq 0, & \forall i, m, p \in \mathcal{P}_{i,m}^{\text{in}}, \\ x_{i,m,q}^{\text{out}} \geq 0, & \forall i, m, q \in \mathcal{P}_{i,m}^{\text{out}}, \\ \alpha^L \geq 0, & \\ 0 \leq U_{\text{link}}^{\text{max}} \leq 1. & \end{array} \right. \quad (28)
 \end{aligned}$$

Here, $\mathbf{P}_L(\bar{y})$ denotes the traffic-allocation problem under a given candidate placement $\bar{y}$. Since $\bar{y}$ is fixed, $\mathbf{P}_L(\bar{y})$ contains no binary placement variables and is therefore a continuous linear program.

After solving the traffic-allocation layer model for all candidate placements, the two-level approximation selects the candidate with the minimum traffic-allocation-layer objective value:

$$r^* = \arg \min_{r=1, \dots, R} F_L^*(y^{(r)}). \quad (29)$$

The final task placement and traffic pre-allocation solution are given by

$$y^* = y^{(r^*)}, \quad x^* = x^*(y^{(r^*)}). \quad (30)$$

E. Discussion

The single-level joint optimization model directly optimizes task placement and traffic allocation, and can fully capture the coupling among node failures, link failures, compute capacities, link capacities, and failure losses. Therefore, it serves as a benchmark for small-scale scenarios. Its main limitation is that the problem size grows rapidly with the number of tasks, candidate paths, and failure scenarios, leading to high computational complexity.

The two-level approximation is not a simple decomposition of the single-level objective. Instead, it is a structural ap-

proximation motivated by the timescale difference in compute routing. The node layer corresponds to slow-timescale task placement and focuses on compute capacity, node risk, and candidate path structures. The link layer corresponds to fast-timescale traffic scheduling and optimizes the actual traffic allocation given a fixed placement. This design avoids frequent changes in task placement caused by network-state variations and reduces the computational burden of the complete joint optimization problem.

In summary, the single-level model provides a complete but heavy joint optimization benchmark, whereas the two-level model offers a scalable approximation framework that better matches practical deployment timescales.

In the evaluation, we will compare the two models in terms of failure loss, node utilization, link utilization, and solving time.